ULPGC
Universidad de
Las Palmas de
Gran Canaria

ESCUELA DE
INGENIERÍA INFORMÁTICA

Ejercicio Individual 1: Respuesta Temporal.

(Sistemas Robóticos Autónomos)

Iván Pérez Díaz

31/10/2026

Índice de Contenidos

1 Fichero .m para la Solución del Ejercicio 2
1.1 Valores de Entrada del Fichero 2
1.2 Cabecera y Entradas 3
1.3 Valor Final 3
1.4 Tiempo de Pico 3
1.5 Frecuencia Amortiguada 4
1.6 Tiempo de Establecimiento 5
1.7 Error en Régimen Permanente 5

2 Modelo y Simulación en Simulink 6
2.1 Diagrama Completo 6
2.2 Bloque Step 7
2.3 Bloque Transfer Fcn 7
2.4 Bloque To workspace 8

3 Script de prueba del fichero .m. 9
3.1 Objetivo del script. 9
3.2 Parámetros y generación de la señal. 9
3.3 Llamada a la función y valores teóricos. 10
3.4 Resultados por pantalla. 11
3.5 Representación gráfica. 11
3.6 Exportación de la figura. 12

1. Fichero .m para la Solución del Ejercicio

1.1. Valores de Entrada del Fichero

El ejercicio propuesto consiste en la creación de un fichero .m que reciba como entrada los valores de dicha señal y los correspondientes instantes de tiempo, y devuelva los siguientes resultados:

- **Frecuencia amortiguada [Hz]:** Calculada como la inversa del pseudo-periodo de la respuesta.
- **Tiempo de establecimiento [seg]:** Calculado como el tiempo que tarda la señal en entrar en +-5 % de su valor final.
- **Tiempo de pico [seg]:** Instante de tiempo en el que se produce la primera sobreoscilación.
- **Error en régimen permanente**

Para la obtención de los resultados, se ha realizado el siguiente código en MatLab, fichero que se puede ver adjunto a la entrega o a través del siguiente enlace: solución-ejercicio-1-Iván-Pérez-Díaz.m.

El objetivo final es simular un sistema similar al de la ***Figura 1***.

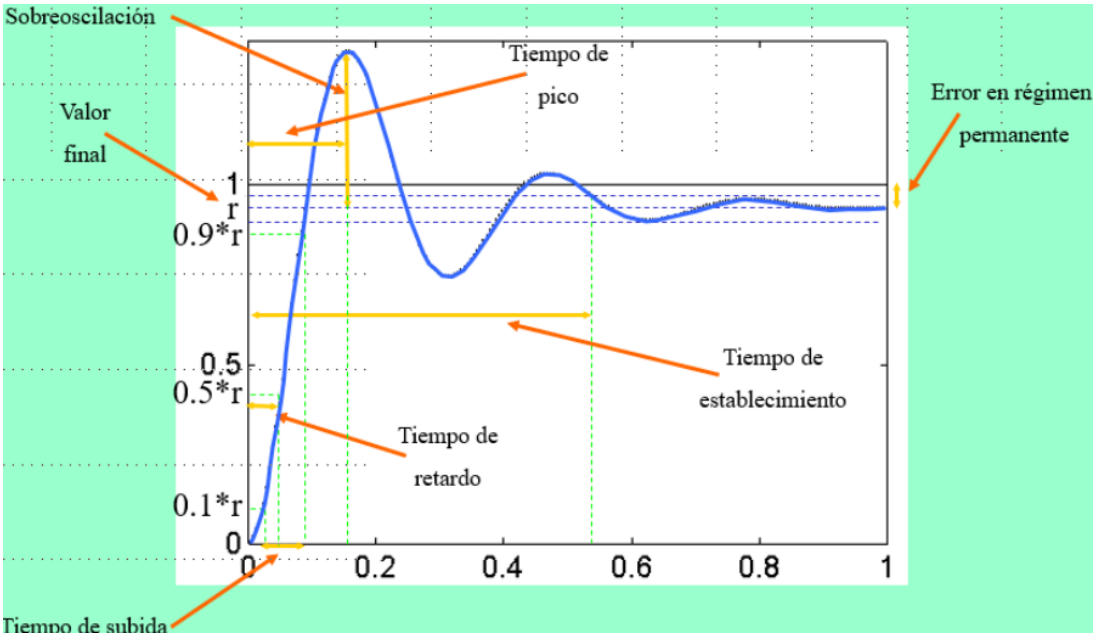


Figura 1. Estado final del sistema deseado.

1.2. Cabecera y Entradas

```
function [frec_amortiguada, temp_establecimiento, temp_pico,
    error_permanente] = sol_ejercicio_1(t, y, r)
```

La función recibe la señal **y** y sus correspondientes instantes de tiempo **t** y devuelve los cuatro resultados pedidos en el enunciado del ejercicio, además, como señal de referencia, se ha añadido la entrada **r** que funciona como referencia del escalón y sirve para calcular el error en régimen permanente.

```
if nargin < 3, r = 1; end
t = t(:); y = y(:);
```

- `nargin`**:** Es el número de argumentos con los que se ha llamado a la función. Si no se pasa `r` se asume un escalón unitario, encajando con la figura en donde la referencia es 1.
- `t(:)` **e** `y(:)`**:** Fuerzan a que ambos sean vectores columna, independientemente de cómo se pasen a la función. Arreglando posibles problemas cuando se use el bloque `To Workspace` de Simulink, que puede devolver vectores fila.

1.3. Valor Final

```
y_final = y(end);
```

Se toma como valor final la última muestra de la señal ya que en una respuesta subamortiguada las oscilaciones se atenuán y la señal se estabiliza, loq ue supone que la simulación dura lo suficiente para que ya esté asentada. Es el ***"valor final"*** que aparece en la ***Figura 1***.

1.4. Tiempo de Pico

Para la detección de máximos locales, se ha programado la siguiente línea de código:

```
idx_picos = find(diff(sign(diff(y))) < 0) + 1;
```

- `diff(y)`: Calcula la diferencia entre muestras consecutivas, es decir, la dervidada discreta. Es positiva si la señal sube y negativa si baja.
- `sign(...)`: Convierte la dereviada discreta en 1 (sube), -1 (baja) o 0 (plana).
- `diff(sign(...))`: Calcula la diferencia entre muestras consecutivas de la derivada discreta. Si la señal pasa de subir a bajar, la diferencia es negativa, y si pasa de bajar a subir, es positiva.
- `find(... <0)`: Devuelve las posiciones donde ocurre un cambio de subida a bajada, es decir, los máximos locales.
- El `+ 1` es necesario porque la función `diff` devuelve un vector con una longitud menor que el original, por lo que hay que compensar el índice. La posición `k` del resultado corresponde con la muestra `k+1` de `y`.

El resultado `idx_picos`, contiene los índices de todos los picos, en orden temporal. El primero es la sobreoscilación principal y el segundo es el siguiente máximo local, mucho más pequeño.

En cuanto a la obtención del tiempo de pico, se ha programado la siguiente línea de código:

```
if ~isempty(idx_picos)
  temp_pico = t(idx_picos(1));
else
  [~, idx] = max(y); temp_pico = t(idx);
end
```

La rama `else` es una protección. Si no se detecta ningún máximo local, se toma el máximo global con `max(y)`. La virgulilla descarta el valor máximo porque solo interesa su índice.

1.5. Frecuencia Amortiguada

```
if numel(idx_picos) >= 2
  T_d = t(idx_picos(2)) - t(idx_picos(1));
  frec_amortiguada = 1 / T_d;
else
  frec_amortiguada = 0;
end
```

El enunciado define la frecuencia amortiguada como la inversa del pseudo-periodo. El pseudo-periodo `T_d` es el tiempo entre dos picos consecutivos, y el código usa los dos primeros. Su inversa está en Hz.

Se exige `numel(idx_picos) >= 2` porque si hay menos de dos picos, no se puede calcular el pseudo-periodo. En ese caso, se devuelve 0.

1.6. Tiempo de Establecimiento

```
banda = 0.05 * abs(y_final);
fuera_banda = find(abs(y - y_final) > banda);
if isempty(fuera_banda)
  temp_establecimiento = t(1);
else
  temp_establecimiento = t(min(fuera_banda(end) + 1, numel(t)));
end
```

El enunciado define el tiempo de establecimiento como el tiempo que tarda la señal en entrar en +-5 % de su valor final y quedarse dentro. En la ***Figura 1*** son las líneas discontinuas alrededor del valor final.

- `banda`**:** Es la semianchura de la franja de +-5 % del valor final. El `abs` evita una banda negativa en el caso de que el valor final fuese negativo.
- `abs(y - y_final) >banda`**:** Marca las muestras que están fuera de la franja, y el `find` devuelve sus índices.
- `fuera_banda(end)`**:** Es la última muestra que está fuera de al franja. Se usa la última y no la primera porque con oscilaciones, la señal puede entrar y salir de la banda varias veces anrtes de asentarse.
- `min(..., numel(t))`**:** Evita salirse del vector si la última muestra aún está fuera de la banda. En ese caso, devuelve el último instante.

1.7. Error en Régimen Permanente

```
error_permanente = abs(r - y_final);
```

2. Modelo y Simulación en Simulink

2.1. Diagrama Completo

Para crear el diagrama, tras guardar el proyecto se ha de ir a **Simulink** y poner los siguientes bloques para poder ejecutar correctamente la simulación y poder ver la onda final:

- **Step:** Simula la entrada escalón (`r`).
- **Transfer Fcn:** Es la simulación del sistema subamortiguado.
- **To Workspace:** Es el espacio de trabajo hecho en MatLab explicado en el **primer punto** de este documento.
- **Scope:** Se usa para ver la señal resultante.

Una vez puestos todos los bloques, el diagrama se ve tal y como se aprecia en la ***Figura 2***.

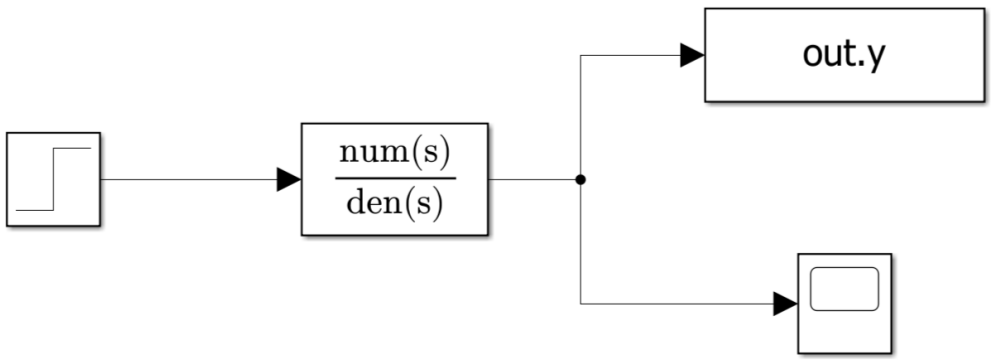


Figura 2. Diagrama en Simulink.

2.2. Bloque Step

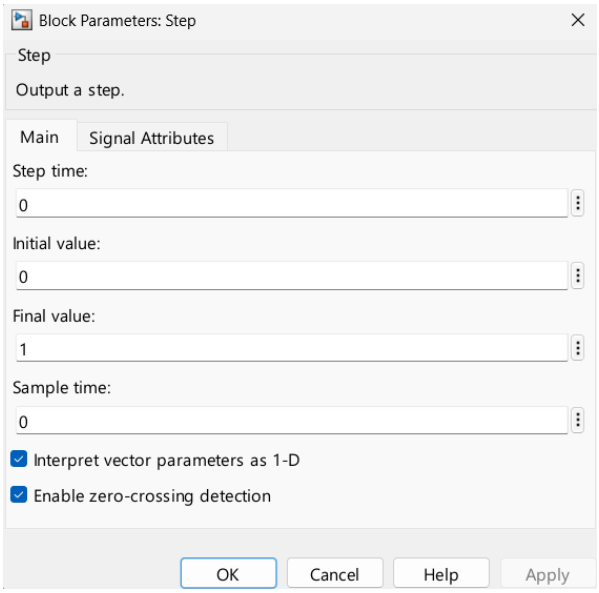


Figura 3. Parámetros del Bloque Step.

2.3. Bloque Transfer Fcn

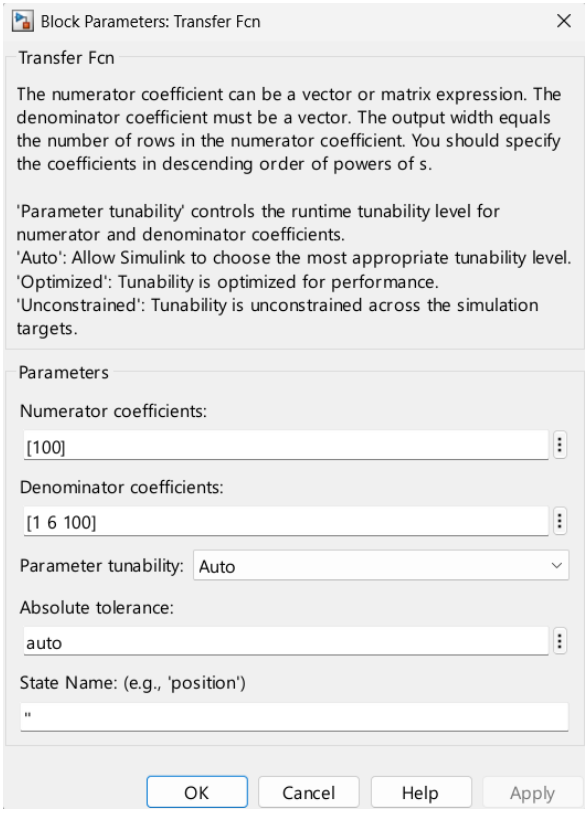


Figura 4. Parámetros del Bloque Transfer Fcn.

2.4. Bloque To workspace

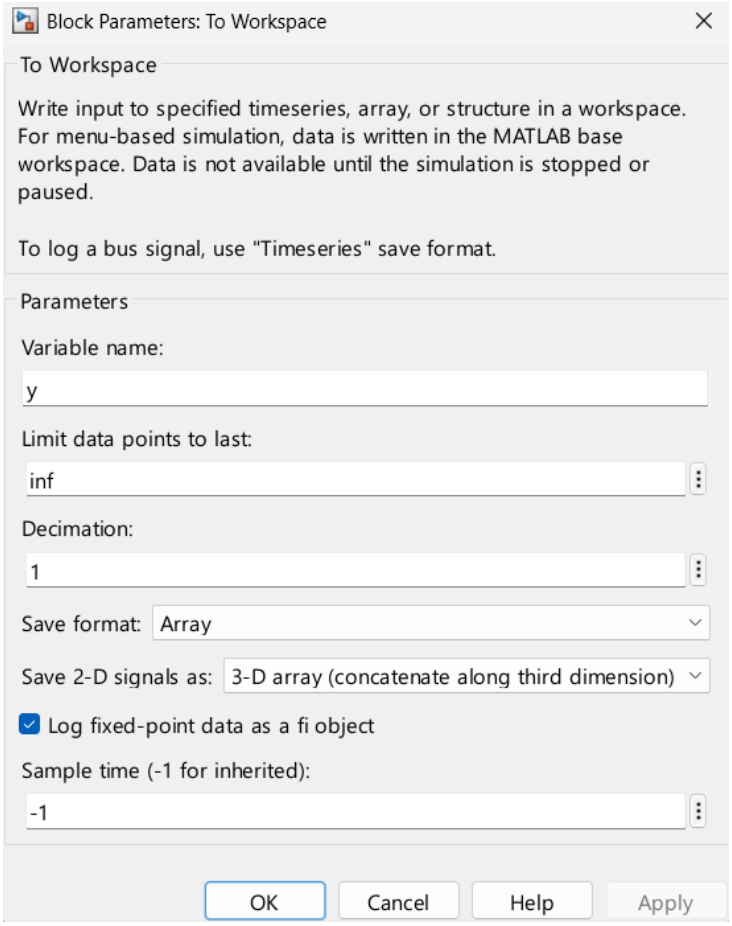


Figura 5. Parámetros del bloque toWorkspace.

3. Script de prueba del fichero .m.

3.1. Objetivo del script.

Para comprobar que la función `sol_ejercicio_1` calcula correctamente los cuatro resultados pedidos, se ha creado el fichero `test_sol_ejercicio_1.m`. El script realiza los siguientes pasos:

- Genera una respuesta temporal subamortiguada de un sistema de segundo orden, de la que se conocen sus valores teóricos.
- Llama a la función con la señal, los instantes de tiempo y la referencia.
- Compara los resultados calculados con los teóricos.
- Dibuja la señal marcando las magnitudes obtenidas y guarda la gráfica en PDF y PNG.

3.2. Parámetros y generación de la señal.

```
r    = 1;        % Referencia del escalón
K    = 0.9;      % Ganancia estática
wn   = 20;       % Frecuencia natural no amortiguada [rad/s]
zeta = 0.3;      % Coeficiente de amortiguamiento

Tm    = 1e-3;    % Paso de muestreo [s]
t_fin = 2;       % Duración de la simulación [s]
```

Se define un sistema de segundo orden con amortiguamiento $\zeta = 0{,}3$, que al ser menor que 1 produce una respuesta subamortiguada, es decir, con oscilaciones que se atenúan. La ganancia estática $K = 0{,}9$ se ha elegido distinta de la referencia para que el sistema se estabilice por debajo de ella y exista un error en régimen permanente medible. La duración de 2 segundos es suficiente para que la señal se asiente, y el paso de 1 ms fija la precisión temporal de los resultados.

```
t    = (0:Tm:t_fin)';
wd   = wn*sqrt(1 - zeta^2);
phi  = acos(zeta);
y    = K*r*(1 - exp(-zeta*wn*t)/sqrt(1 - zeta^2) .* sin(wd*t + phi));
```

La señal se genera con la expresión analítica de la respuesta al escalón de un sistema de segundo orden subamortiguado:

$$y(t) = K\,r\left(1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1-\zeta^2}}\,\sin(\omega_d t + \varphi)\right), \quad \omega_d = \omega_n\sqrt{1-\zeta^2}, \quad \varphi = \arccos(\zeta)$$

- `t`: Vector columna de instantes de tiempo, desde 0 hasta `t_fin` con paso `Tm`. La comilla final lo traspone para que sea columna, como espera la función.
- `wd`: Frecuencia natural amortiguada ω_d en rad/s.
- `phi`: Desfase φ de la senoidal.
- `y`: Señal resultante. El operador `.*` multiplica elemento a elemento los vectores del exponencial y el seno.

Si en lugar de esta señal sintética se quisiera usar la obtenida con Simulink, bastaría sustituir estas líneas por la lectura de las variables que genera el bloque `To Workspace` (por ejemplo, `t = y2.Time; y = y2.Data;` si se guarda como `Timeseries`).

3.3. Llamada a la función y valores teóricos.

```
[frec_amortiguada, temp_establecimiento, temp_pico, error_permanente] =
    ...
    sol_ejercicio_1(t, y, r);

fd_teo  = wd/(2*pi);
tp_teo  = pi/wd;
ts_teo  = -log(0.05*sqrt(1 - zeta^2))/(zeta*wn);
ess_teo = abs(r - K*r);
```

Primero se invoca la función con la señal, el tiempo y la referencia, y se guardan los cuatro resultados. A continuación se calculan los valores teóricos con las fórmulas conocidas de un sistema de segundo orden:

- **Frecuencia amortiguada:** $f_d = \omega_d/2\pi$, que es la inversa del pseudo-periodo.
- **Tiempo de pico:** $t_p = \pi/\omega_d$, instante de la primera sobreoscilación.

- **Tiempo de establecimiento (5 %):** se obtiene igualando la envolvente exponencial de las oscilaciones al 5 %, por lo que es una aproximación.
- **Error permanente:** $|r - Kr|$, diferencia entre la referencia y el valor final al que tiende el sistema.

3.4. Resultados por pantalla.

```
fprintf('%-30s %12s %12s\n', 'Magnitud', 'Calculado', 'Teorico');
fprintf('%-30s %12.4f %12.4f\n', 'Frecuencia amortiguada [Hz]', ...
        frec_amortiguada, fd_teo);
% ... (una línea equivalente para cada resultado)
```

Se muestra una tabla por la consola de MATLAB con cada magnitud calculada junto a su valor teórico. El formato `%-30s` alinea el texto a la izquierda en 30 caracteres y `%12.4f` escribe los números con 4 decimales. Los resultados obtenidos se recogen en la Tabla 1.

Magnitud	Calculado	Teórico
Frecuencia amortiguada [Hz]	3.0395	3.0365
Tiempo de establecimiento [s]	0.5070	0.5071
Tiempo de pico [s]	0.1650	0.1647
Error en régimen permanente	0.1000	0.1000

Tabla 1. Resultados calculados por la función y valores teóricos.

Los valores calculados coinciden con los teóricos salvo diferencias del orden de la milésima. Esto es lo esperado, porque la función solo puede devolver instantes que coinciden con una muestra, y el paso de muestreo es de 1 ms. En el caso del tiempo de establecimiento, el valor teórico es además una aproximación basada en la envolvente.

3.5. Representación gráfica.

```
y_final = y(end);
[~, ip] = min(abs(t - temp_pico));
y_pico  = y(ip);

plot(t, y, 'b', 'LineWidth', 1.8); hold on; grid on;
```

```
plot([t(1) t(end)], [r r], 'k-');
plot([t(1) t(end)], [y_final y_final], 'k--');
plot([t(1) t(end)], 1.05*y_final*[1 1], 'b:');
plot([t(1) t(end)], 0.95*y_final*[1 1], 'b:');
plot(temp_pico, y_pico, 'ro', 'MarkerFaceColor', 'r');
plot([temp_establecimiento temp_establecimiento], ...
     [0 1.05*y_final], 'g--');
plot([t(end) t(end)], [y_final r], 'm-', 'LineWidth', 3);
```

El script dibuja la señal y superpone sobre ella las magnitudes calculadas, de forma similar a la figura del enunciado:

- `min(abs(t - temp_pico))`: Busca la muestra más cercana al tiempo de pico para obtener su altura `y_pico`.
- **Línea negra continua:** Referencia r.
- **Línea negra discontinua:** Valor final de la señal.
- **Líneas azules punteadas:** Banda de $\pm 5\,\%$ alrededor del valor final.
- **Punto rojo:** Primera sobreoscilación, en el instante del tiempo de pico.
- **Línea vertical verde:** Tiempo de establecimiento, a partir del cual la señal queda dentro de la banda.
- **Segmento magenta:** Error en régimen permanente, entre el valor final y la referencia.

El resultado se muestra en la ***Figura 6***.

Falta la imagen `images/respuesta-temporal.png`

Figura 6. Respuesta temporal de prueba con los resultados calculados.

3.6. Exportación de la figura.

```
try
    exportgraphics(fig, 'respuesta_temporal.pdf', 'ContentType', 'vector
        ');
    exportgraphics(fig, 'respuesta_temporal.png', 'Resolution', 300);
```

```
catch
    saveas(fig, 'respuesta_temporal.png');
    print(fig, 'respuesta_temporal.pdf', '-dpdf', '-bestfit');
end
```

Por último, la gráfica se guarda en dos formatos: PDF vectorial, que es el que pide la entrega, y PNG con 300 ppp para incluirlo en este documento. El bloque `try/catch` intenta usar `exportgraphics`, disponible en versiones recientes de MATLAB, y, si falla, recurre a `saveas` y `print`, que funcionan en versiones antiguas.